CAPACITACIÓN QA AUTOMATION · PARTE 1 DE 2

SÍLABO · 27 AL 30 DE OCTUBRE DE 2026

Pruebas de APIs con REST Assured + Cucumber

Cuatro sesiones de 2 horas, 100 % prácticas: cada una termina con código tuyo en verde y un Pull Request. El viernes, pipeline de CI con reportes y examen final.

Java 17	REST Assured 6	Cucumber 7	GitHub Actions
Temurin LTS	cliente HTTP	Gherkin en español	CI y reportes

Al terminar la semana sabrás

1. Leer la documentación de una API y convertirla en escenarios Gherkin en español.
2. Automatizar consultas, altas, cambios y bajas con autenticación por token.
3. Validar códigos, cuerpos, tiempos y el **esquema JSON** de las respuestas.
4. Organizar el proyecto para que crezca: especificaciones reutilizables, ambientes y etiquetas.
5. Generar **reportes** y correr todo en un **pipeline de CI** en cada Pull Request.

Instructor: **Giancarlo Palomino** · QA Automation Engineer

Mar 27 a vie 30 · 15:00 a 17:00 · 100 % práctico

PLAN DE SESIONES

Martes y miércoles

1 MARTES 27 — Del HTTP al primer escenario

15 min	Entorno verificado: `mvn test` en verde en todas las PC
20 min	Qué es una API: petición, respuesta, verbos, códigos de estado y JSON
25 min	Gherkin y el proyecto por dentro: feature, steps, runner, hooks y contexto
30 min	En vivo: consultar reservas con filtros y pedir una que no existe
30 min	Ejercicios 1 y 2

EJERCICIOS

- **1** · Buscar reservas por nombre y pedir un id inexistente (404)
- **2** · Crear una reserva con una tabla de datos y verificar cada campo

ENTREGA

PR `feature/s1-<tu-nombre>`

2 MIÉRCOLES 28 — Autenticación y ciclo de vida completo

10 min	Revisión de los PR de ayer
30 min	Token con `POST /auth`, cabeceras, PUT, PATCH y DELETE
20 min	Objetos Java en lugar de mapas: serialización con Jackson
20 min	Validar la forma de la respuesta con JSON Schema
40 min	Ejercicios 3 y 4

EJERCICIOS

- **3** · Autenticarse y actualizar con token (y fallar sin él)
- **4** · Ciclo de vida: crear, consultar, actualizar, eliminar y comprobar

ENTREGA

PR `feature/s2-<tu-nombre>`

PLAN DE SESIONES

Jueves y viernes

3 JUEVES 29 — Código que escala y caza de bugs

35 min	RequestSpecification, cliente por recurso, URL por ambiente, log si falla
15 min	Esquema del escenario con negativos; etiquetas @smoke y @regresion
20 min	Reportes de Cucumber: HTML, JSON y JUnit; cómo leer un fallo
50 min	Ejercicio 5

EJERCICIOS

- **5** · Caza de bugs: la API esconde al menos cinco

ENTREGA

PR `feature/s3-<tu-nombre>` + `BUGS.md`

4 VIERNES 30 — Reportes, pipeline y examen final

40 min	GitHub Actions: disparadores, caché, reporte como artefacto, resumen en el PR y GitHub Pages
10 min	@smoke en cada PR y regresión completa programada
60 min	Examen final sobre la API del Portal Canvia Market
10 min	Cierre y siguientes pasos

EJERCICIOS

- Examen final (60 min)

ENTREGA

PR `feature/examen-<tu-nombre>`

VIERNES 30 · 60 MINUTOS

Examen final sobre el Portal Canvia Market

Automatizas contra la API del Portal Canvia Market (catálogo y pedidos con autenticación), en tu rama `feature/examen-<tu-nombre>`, con el pipeline levantando el portal y publicando el reporte.

CRITERIO	PESO
Escenarios en Gherkin claros, en español y declarativos	**20 %**
Autenticación, CRUD y casos negativos funcionando	**30 %**
Validaciones correctas (código, cuerpo y esquema)	**20 %**
Pipeline en verde con el reporte publicado	**20 %**
Bugs encontrados y documentados	**10 %**

APROBACIÓN

70 %. Mínimo: 8 escenarios que cubran autenticación, CRUD, pedidos y pipeline. El enunciado completo se publica en el portal (`/examen`) al iniciar.

Cómo se evalúa cada sesión

	CUMPLE	FALTA
Ejecuta	`mvn test` en verde en tu PC y en la CI	Falla o no compila
Se entiende	Gherkin de negocio, sin URLs ni JSON en el feature	Pasos técnicos en el escenario
Valida de verdad	Comprueba código **y** contenido	Solo mira el 200
Es independiente	Cada escenario crea sus datos	Ids fijos o dependencia del orden

REQUISITO

Instalación completa: `mvn test` en verde (manual de instalación o entorno Docker).

CÓMO SE ENTREGA

Una rama `feature/s<N>-<tu-nombre>` por sesión y un Pull Request a `develop` con la CI en verde. Ver el manual de Git y GitHub.